ENTORNOS DE DESARROLLO · 1º DAM / DAW

UD1

Desarrollo de software

RA1. Reconoce los elementos y herramientas que intervienen en el desarrollo de un programa informático, analizando sus características y las fases en las que actúan hasta llegar a su puesta en funcionamiento.

Cómo usar estos apuntes

- Las cajas **DEFINICIÓN** recogen los conceptos que debes saber enunciar con precisión.
- Las cajas **IDEA CLAVE** resumen lo más importante de cada apartado.
- Al final de cada bloque hay un **glosario** y unas **preguntas de autoevaluación**.
- Al final del documento tienes la correspondencia con los criterios de evaluación (CE1.a – CE1.g).

Índice

1. El ordenador y el software 3

2. Programas y aplicaciones informáticas 5

3. Lenguajes de programación 8

4. Traducción y compilación de código 13

5. Fases del desarrollo de una aplicación 17

Correspondencia con los criterios de evaluación 25

1. El ordenador y el software

1.1. ¿Qué es un ordenador?

> DEFINICIÓN · **Ordenador:** máquina que acepta datos de entrada, los procesa siguiendo unas instrucciones y produce unas salidas (resultados).

Un ordenador se compone de elementos físicos (**hardware**) y elementos lógicos (**software**). Como el ordenador solo entiende señales eléctricas, las órdenes del usuario deben ser traducidas a un formato que pueda procesar.

Los primeros ordenadores aparecen en la década de 1940 (ENIAC, la máquina de Turing…).

1.2. Componentes de un ordenador

> DEFINICIÓN · **Hardware:** conjunto de elementos físicos (tangibles) de un sistema informático.

Componente	Función
Procesador (CPU)	Ejecuta las instrucciones de los programas. Contiene la **Unidad de Control** (dirige la ejecución) y la **Unidad Aritmético-Lógica, ALU** (realiza operaciones aritméticas y lógicas).
Memoria principal (RAM)	Almacena temporalmente los programas y los datos mientras se están ejecutando.
Periféricos de entrada/salida	Dispositivos que comunican el ordenador con el usuario: teclado, ratón (entrada); pantalla, impresora (salida).

1.3. ¿Qué es el software?

> DEFINICIÓN · **Software:** parte intangible (lógica) de un sistema informático: conjunto de programas e instrucciones que indican al hardware qué debe hacer.

- El software **se desarrolla**; no se fabrica como un elemento físico.
- Por lo general **no se estropea**, aunque puede contener errores.
- Se desarrolla usando un **lenguaje de programación** y una **metodología de trabajo**.

1.4. Relación hardware-software

Hardware y software están estrechamente relacionados: uno no funciona sin el otro. El **sistema operativo** actúa de intermediario y gestiona el acceso de las aplicaciones a la CPU, la memoria RAM y los periféricos.

> DEFINICIÓN · **Sistema operativo:** software que gestiona los recursos del hardware y sirve de intermediario entre las aplicaciones y el equipo.

> **IDEA CLAVE** · El hardware solo entiende **código binario** (señales eléctricas). Por eso el software escrito en un lenguaje de programación debe **traducirse** antes de ejecutarse.

1.5. Arquitectura de Von Neumann

> **DEFINICIÓN · Arquitectura de Von Neumann:** modelo de ordenador (1946) basado en el **programa almacenado**: las instrucciones y los datos comparten la misma memoria.

Sus tres bloques son la **memoria principal**, la **CPU** (Unidad de Control + ALU) y los **periféricos de E/S**, comunicados entre sí. La CPU procesa cada instrucción repitiendo sin parar el **ciclo de instrucción**:

Paso	Fase	Qué ocurre
1	Captación	La Unidad de Control busca en la memoria la siguiente instrucción.
2	Decodificación	La Unidad de Control interpreta qué operación hay que realizar.
3	Ejecución	La ALU realiza la operación: un cálculo o una comparación.
4	Almacenamiento	El resultado vuelve a la memoria o sale por un periférico, y el ciclo se repite.

Glosario del bloque

Término	Definición
Ordenador	Máquina que recibe datos, los procesa con instrucciones y produce resultados.
Hardware	Parte física del sistema informático.
Software	Parte lógica (intangible): programas y datos.
CPU	Procesador: ejecuta las instrucciones. Incluye UC y ALU.
Unidad de Control (UC)	Dirige y coordina la ejecución de las instrucciones.
ALU	Unidad Aritmético-Lógica: realiza operaciones aritméticas y lógicas.
RAM	Memoria principal: guarda temporalmente programas y datos en ejecución.
Periférico	Dispositivo de entrada o salida que comunica el sistema con el usuario.
Sistema operativo	Software intermediario entre aplicaciones y hardware.
Código binario	Lenguaje de ceros y unos que entiende el hardware.
Programa almacenado	Idea de Von Neumann: instrucciones y datos en la misma memoria.
Ciclo de instrucción	Captación → decodificación → ejecución → almacenamiento.

Para comprobar lo aprendido

1. ¿Qué diferencia hay entre hardware y software? Pon un ejemplo de cada uno.
2. ¿Qué dos unidades componen la CPU y qué hace cada una?
3. Explica el concepto de «programa almacenado».
4. Enumera, en orden, las cuatro fases del ciclo de instrucción.

2. Programas y aplicaciones informáticas

2.1. Programas y aplicaciones

> DEFINICIÓN · **Programa (o aplicación):** conjunto de instrucciones que indican al ordenador cómo realizar un proceso.

Se desarrolla usando un lenguaje de programación. Si el ordenador no entiende una instrucción, lo notifica mediante un mensaje de error. Ejemplos: sistemas operativos, ofimática, bases de datos, correo, mensajería, videojuegos…

2.2. Software a medida vs. software estándar

> DEFINICIÓN · **Software a medida:** software desarrollado según los requisitos de una empresa u organismo concreto.
>
> DEFINICIÓN · **Software estándar:** software genérico, válido para cualquier cliente, que se compra ya desarrollado y solo se configura.

	A medida	Estándar
Desarrollo	Según los requisitos de la empresa; se adapta a su actividad.	Genérico, válido para cualquier cliente.
Adquisición	Necesita tiempo de desarrollo y mantenimiento propio.	Se compra ya desarrollado; solo se configura.
Coste	Suele ser más costoso.	Normalmente más barato.
Fiabilidad	Menos probado por tener menos usuarios.	Más probado → suele tener menos errores.
Ajuste	Hace justo lo que se necesita.	Puede incluir funciones que no se usan o carecer de otras necesarias.

2.3. Herramientas y conceptos básicos

> DEFINICIÓN · **Librería (o biblioteca):** conjunto de funciones o clases ya programadas que un programa puede reutilizar (por ejemplo, `java.util`, `java.io`).
>
> DEFINICIÓN · **Entorno de desarrollo (IDE):** herramienta que facilita y posibilita el desarrollo de software (Eclipse, VS Code…).

2.4. Entornos integrados de desarrollo (IDE)

> DEFINICIÓN · **IDE (Entorno Integrado de Desarrollo):** software que reúne, en una sola interfaz, las herramientas necesarias para programar.

Un IDE se compone de: **editor de código**, **compilador y/o intérprete**, **depurador** y, a menudo, un **constructor de interfaces gráficas**. Frente a usar un editor de texto simple y la terminal por separado, un IDE gana en fiabilidad y ahorra tiempo. Suelen estar orientados a uno o varios lenguajes y son ampliables mediante **plugins** (Eclipse, NetBeans, Visual Studio…).

2.5. Funciones de un entorno de desarrollo

Función	Descripción
Editor inteligente	Coloreado de sintaxis, autocompletado de código y detección de errores en tiempo real.
Depurador	Permite seguir el valor de las variables y detener la ejecución en puntos concretos.
Control de versiones	Guarda un histórico de cambios y permite recuperar versiones anteriores del proyecto.
Refactorización	Facilita cambios en el código (por ejemplo, renombrar) sin alterar su funcionamiento.
Documentación	Genera documentación automáticamente a partir de los comentarios del código.
Gestión de proyectos	Organiza archivos fuente, compilados y recursos dentro de una misma interfaz.

2.6. Comparativa de entornos de desarrollo

IDE	Lenguajes	Licencia	Uso recomendado
NetBeans	C/C++, Java, JavaScript, PHP, Python	Libre	Multiplataforma en Java
Eclipse	Ada, C/C++, Java, JavaScript, PHP	Libre	Proyectos grandes en Java
Microsoft Visual Studio	Basic, C/C++, C#	Propietaria	Ecosistema Windows/.NET
C++ Builder	C/C++	Propietaria	Escritorio en C++
JBuilder	Java	Propietaria	Apps empresariales (uso histórico)

La elección del IDE depende del **lenguaje** a utilizar y del **tipo de licencia** con la que se quiera trabajar.

2.7. Herramientas CASE

> **DEFINICIÓN · CASE (Computer-Aided Software Engineering):** herramientas de ingeniería del software asistida por ordenador que automatizan tareas del desarrollo para ganar tiempo y fiabilidad.

Tipo	Ayuda en…
U-CASE	Planificación y análisis
M-CASE	Diseño
L-CASE	Programación, pruebas y documentación
I-CASE	CASE integradas: combinan varias de las fases anteriores en una misma herramienta

Ventajas: reducción de errores humanos, documentación más completa y mayor coherencia entre las fases. Ejemplos: StarUML y Visual Paradigm (diagramas de diseño), SonarQube (calidad de código).

2.8. Desarrollo rápido de aplicaciones (RAD)

> **DEFINICIÓN · RAD (Rapid Application Development):** forma de desarrollar software rápido: en lugar de planificarlo todo antes de programar, se construyen **prototipos funcionales** desde el principio que se enseñan al cliente y se mejoran poco a poco hasta llegar al programa final.

Combina desarrollo **iterativo**, **prototipos** y **herramientas CASE**. Su objetivo es reducir el tiempo de desarrollo implicando al cliente en revisiones frecuentes. Hoy el término también se asocia al desarrollo rápido de interfaces gráficas o a entornos de desarrollo completos.

Glosario del bloque

Término	Definición
Programa / aplicación	Conjunto de instrucciones que realizan un proceso.
Software a medida	Desarrollado para un cliente concreto.
Software estándar	Genérico, válido para cualquier cliente.
Librería	Funciones o clases ya programadas, reutilizables.
IDE	Entorno que integra editor, compilador/intérprete y depurador.
Depurador	Herramienta para seguir variables y detener la ejecución y encontrar errores.
Plugin	Extensión que añade funciones a un IDE.
Control de versiones	Histórico de cambios que permite recuperar versiones anteriores.
Refactorización	Cambiar el código sin alterar su funcionamiento.
CASE	Herramientas que automatizan tareas del desarrollo.
RAD	Desarrollo rápido mediante prototipos y revisiones con el cliente.
Licencia libre / propietaria	Libre: se puede estudiar y modificar. Propietaria: uso restringido.

Para comprobar lo aprendido

1. Compara software a medida y software estándar con dos ventajas y dos inconvenientes de cada uno.
2. ¿Qué componentes forman un IDE?
3. ¿Qué significan las siglas CASE y qué tipos hay?
4. Define RAD y explica qué papel tiene el cliente.

3. Lenguajes de programación

3.1. ¿Qué es un lenguaje de programación?

> DEFINICIÓN · **Lenguaje de programación:** lenguaje artificial, creado por programadores, que permite traducir las instrucciones de un programa a código comprensible por el ordenador.

Se compone de un conjunto de símbolos (**sintaxis**) y de reglas (**semántica**). Es mucho más cercano al lenguaje humano que el lenguaje máquina (0 1 0 1…). Existen muchos, cada uno con sus propias características (Java, Python, C++, JavaScript…).

> DEFINICIÓN · **Sintaxis:** conjunto de símbolos y reglas de escritura válidas de un lenguaje.
>
> DEFINICIÓN · **Semántica:** significado de las instrucciones: qué hace cada una al ejecutarse.

3.2. Clasificación según su nivel

Nivel	Descripción	Ejemplos
Lenguaje máquina	Código binario (0/1). Es el único que entiende directamente el ordenador.	Instrucciones en binario
Medio nivel (ensamblador)	Usa mnemotécnicos cercanos al hardware.	ADD, MOV
Alto nivel	Cercano al lenguaje humano.	C, Java, Python, JavaScript

3.3. Paradigmas de programación

> DEFINICIÓN · **Paradigma de programación:** estilo o forma de pensar y organizar un programa.

Se agrupan en dos grandes familias: *imperativos* (dicen *cómo* hacerlo, paso a paso) y *declarativos* (dicen *qué* se quiere conseguir).

Paradigma	Idea principal	Ejemplos
Estructurada (imperativa)	Secuencia ordenada de instrucciones (secuencia, selección, iteración), organizada en funciones o procedimientos.	C, Pascal
Orientada a objetos (imperativa)	Representa entidades del mundo real mediante clases y objetos, que combinan datos y comportamiento.	Java, Python
Funcional (declarativa)	El programa se descompone en funciones matemáticas sin efectos secundarios.	Haskell, Lisp
Lógica (declarativa)	Se describen hechos y reglas, y el sistema infiere la solución. Muy usada en IA.	Prolog

3.4. Ejemplo de programación estructurada: C

Creado en 1972, sigue muy vigente en sistemas operativos, software embebido y programación de bajo nivel. El programa es una secuencia de instrucciones organizada en funciones y estructuras de control (bucles, condicionales).

```c
#include <stdio.h>
int main() {
    for (int i = 1; i <= 5; i++)
        printf("%d\n", i);
}
```

- **Secuencia:** las instrucciones se ejecutan una detrás de otra.
- **Iteración:** el bucle `for` repite el bloque mientras `i <= 5` sea cierto.
- **Función:** `main()` es el punto de entrada del programa.
- **Resultado:** imprime los números del 1 al 5, uno por línea.

3.5. Ejemplo de programación orientada a objetos: Java

Creado en 1995, es uno de los lenguajes más usados: apps Android, backend empresarial y sistemas multiplataforma.

```java
class Perro {
    String nombre;
    void ladrar() {
        System.out.println(nombre + ": ¡Guau!");
    }
}
```

> **DEFINICIÓN · Clase:** plantilla que define los datos y el comportamiento de los objetos.
>
> **DEFINICIÓN · Objeto:** instancia concreta de una clase (por ejemplo, creado con `new Perro()`).

- **Atributo:** `String nombre;` guarda el estado propio de cada objeto.
- **Método:** `ladrar()` es una acción que puede realizar cualquier objeto de la clase.

3.6. Ejemplo de programación lógica: Prolog

Surgió en 1972 y se usa en inteligencia artificial: sistemas expertos, procesamiento del lenguaje natural y resolución de problemas lógicos. No se dan instrucciones paso a paso: se describen hechos y reglas.

```
padre(juan, maria).
padre(juan, pedro).
padre(pedro, ana).
abuelo(X, Z) :- padre(X, Y), padre(Y, Z).
```

- **Hecho:** afirma algo cierto (`padre(juan, maria).` = Juan es padre de María).
- **Regla:** define una relación a partir de hechos (`abuelo`).
- **Consulta:** `?- abuelo(juan, ana).` → responde `true`, porque Juan es padre de Pedro y Pedro de Ana.

3.7. Ejemplo de programación funcional: Lisp

Creado en 1958, fue clave en los inicios de la IA; hoy perviven dialectos como Clojure o Emacs Lisp. Aquí una función se llama a sí misma (recursión) en lugar de usar un bucle.

```
(defun factorial (n)
  (if (<= n 1)
      1
      (* n (factorial (- n 1)))))
(factorial 5)
```

> **DEFINICIÓN · Recursión:** técnica en la que una función se llama a sí misma hasta llegar a un caso base.

- **Caso base:** si `n <= 1` devuelve 1 y la recursión se detiene.
- **Caso recursivo:** multiplica n por `factorial (n-1)`.
- **Resultado:** `(factorial 5)` devuelve 120 (5×4×3×2×1).

3.8. Lisp: funciones de orden superior

En Lisp las funciones también se pueden pasar como datos. Aquí `mapcar` aplica una función a cada elemento de una lista.

```
(mapcar (lambda (x) (* x x))
        '(1 2 3 4))
;; devuelve (1 4 9 16)
```

> **DEFINICIÓN · Función de orden superior:** función que recibe otras funciones como argumento o devuelve una función como resultado (como `mapcar`).

- **`mapcar`:** aplica una función a cada elemento de una lista y devuelve una lista nueva.
- **`lambda`:** `(lambda (x) (* x x))` es una función sin nombre que calcula el cuadrado de `x`.
- **Resultado:** la lista `(1 4 9 16)`, sin usar ningún bucle.
- **¿Cuándo se usa?** Para transformar listas; es el uso típico de las funciones de orden superior.

3.9. Anidación de funciones (funcional)

La **anidación de funciones** es una parte de la programación funcional: el resultado de una llamada se pasa directamente como argumento de otra. Aquí la vemos en Python, aunque se puede usar en cualquier lenguaje.

```
def doble(x):
    return x * 2
def suma(a, b):
    return a + b
print(suma(doble(3), doble(4)))
```

> **DEFINICIÓN · Anidación de funciones:** llamar a una función dentro de los argumentos de otra: se ejecutan primero las llamadas internas y su resultado se usa como argumento de la externa.

- **Funciones:** `doble(x)` y `suma(a, b)` devuelven un valor con `return`.

- **Orden de ejecución:** primero `doble(3)` → 6 y `doble(4)` → 8; después `suma(6, 8)`.
- **Resultado:** el programa imprime 14.
- **¿Cuándo se usa?** Para combinar funciones pequeñas en operaciones complejas; es la base del enfoque funcional.

3.10. Código fuente, código objeto y ejecutable

Tipo	Definición
Código fuente	El que escribe el programador en un lenguaje de programación.
Código objeto	Código máquina generado tras compilar el código fuente.
Ejecutable	Programa final, listo para ejecutarse (incluye las librerías ya enlazadas).

3.11. Características deseables del código

Característica	Significado
Modularidad	Dividido en partes más pequeñas y manejables.
Corrección	Hace exactamente lo que se le pide.
Legibilidad	Es fácil de leer, lo que facilita su mantenimiento.
Eficiencia	Aprovecha bien los recursos del sistema.
Portabilidad	Puede ejecutarse en distintos equipos.

Según su licencia, el código fuente puede ser **abierto** (cualquiera puede estudiarlo o modificarlo) o **cerrado** (de uso restringido).

Glosario del bloque

Término	Definición
Lenguaje de programación	Lenguaje artificial para escribir programas.
Sintaxis / semántica	Reglas de escritura / significado de las instrucciones.
Lenguaje máquina	Código binario; el único que entiende la CPU.
Ensamblador	Lenguaje de medio nivel con mnemotécnicos (ADD, MOV).
Lenguaje de alto nivel	Cercano al lenguaje humano (C, Java, Python…).
Paradigma	Estilo de organizar un programa.
Imperativo / declarativo	Dice cómo hacerlo / dice qué se quiere conseguir.
Programación estructurada	Secuencia, selección e iteración con funciones.
Programación orientada a objetos	Basada en clases y objetos.

Término	Definición
Programación funcional	Basada en funciones matemáticas sin efectos secundarios.
Función de orden superior	Función que recibe o devuelve otras funciones (ej. mapcar en Lisp).
Anidación de funciones	Usar el resultado de una llamada como argumento de otra.
Programación lógica	Basada en hechos y reglas.
Código fuente / objeto / ejecutable	Escrito por el programador / código máquina tras compilar / programa final.
Modularidad, portabilidad…	Cualidades deseables del código.

Para comprobar lo aprendido

1. Ordena de menor a mayor cercanía al lenguaje humano: alto nivel, lenguaje máquina, ensamblador.
2. Diferencia paradigma imperativo y declarativo y cita un lenguaje de cada paradigma.
3. ¿Qué son una clase y un objeto?
4. ¿En qué se diferencian código fuente, código objeto y ejecutable?

4. Traducción y compilación de código

4.1. Traductores de código

> **DEFINICIÓN · Traductor:** programa que convierte un lenguaje de alto nivel en ensamblador o lenguaje máquina.

Hay dos tipos principales:

- **Intérprete:** traduce y ejecuta el código línea a línea, «al vuelo».
- **Compilador:** traduce todo el programa antes de poder ejecutarlo.

4.2. El proceso de compilación

> **DEFINICIÓN · Compilación:** proceso que transforma el código fuente escrito por el programador en un programa ejecutable, pasando por varias fases sucesivas.

Paso	Fase	Qué ocurre
1	Código fuente	Archivo escrito por el programador (por ejemplo, `hola.c`).
2	Preprocesado	Se procesan las directivas (`#include`, `#define`) y se eliminan los comentarios.
3	Compilación	Se comprueba la sintaxis y se traduce a código máquina. Aquí aparecen los **errores de compilación**.
4	Código objeto	Archivo con código máquina (`hola.o`), aún incompleto: faltan las librerías.
5	Enlazado	El **enlazador** une el código objeto con las librerías y genera el ejecutable.

> **IDEA CLAVE ·** El resultado es un **ejecutable independiente**: se puede ejecutar sin necesidad del compilador.

4.3. Ejemplo: compilar un programa en C

Con el compilador `gcc` se puede recorrer el proceso paso a paso. Partimos del archivo `hola.c`:

```c
#include <stdio.h>
int main() {
    printf("Hola\n");
}
```

Paso	Comando	Resultado
Preprocesar y compilar	`gcc -c hola.c`	`hola.o` (código objeto)
Enlazar	`gcc hola.o -o hola`	`hola` (ejecutable)
Ejecutar	`./hola`	Muestra «Hola» en pantalla
Todo de una vez	`gcc hola.c -o hola`	`hola` (ejecutable)

4.4. El proceso de interpretación

> **DEFINICIÓN · Intérprete:** traductor que analiza, traduce y ejecuta el programa instrucción a instrucción, sin generar un ejecutable.

Paso	Fase	Qué ocurre
1	Leer	El intérprete lee la siguiente línea del código fuente.
2	Analizar y traducir	Comprueba la sintaxis y traduce solo esa instrucción.
3	Ejecutar	La instrucción se ejecuta de inmediato.
4	¿Más líneas?	Si las hay, se vuelve al paso 1. Si no, el programa termina.

- **Si hay un error,** la ejecución se detiene en esa línea: el error aparece al llegar a ella, no antes.
- **Ejemplos:** PHP, JavaScript, Bash.

4.5. Compilación frente a interpretación

Aspecto	Compilador	Intérprete
Cuándo traduce	Todo el programa, antes de ejecutarlo	Línea a línea, durante la ejecución
Resultado	Genera un ejecutable independiente	No genera ejecutable propio
Velocidad	Más rápido al ejecutarse	Más lento: traduce cada vez
Errores	Se detectan antes de ejecutar	Aparecen al llegar a esa línea
En el equipo	Basta con el ejecutable	Hace falta el intérprete instalado
Ejemplos	C, C++	PHP, JavaScript

4.6. Lenguajes compilados, interpretados y virtuales

Tipo	Funcionamiento	Ejemplo
Compilados	Se traducen por completo antes de ejecutar, generando un ejecutable independiente. Más rápidos, pero necesitan compilador y enlazador.	C/C++
Interpretados	Se traducen y ejecutan línea a línea, sin generar ejecutable propio. El intérprete debe estar cargado en memoria durante toda la ejecución.	PHP
Virtuales	Se compilan a un código intermedio (bytecode), independiente del sistema, que ejecuta una máquina virtual instalada en el equipo.	Java

4.7. Código intermedio y máquinas virtuales

> **DEFINICIÓN · Código intermedio (bytecode):** código generado tras la compilación que no es código máquina directo y que ejecuta una máquina virtual.

> **DEFINICIÓN · Máquina virtual:** programa instalado en el equipo que ejecuta el bytecode (por ejemplo, la JVM de Java).

- **Ventaja:** portabilidad; el mismo bytecode funciona en distintos sistemas sin recompilar.
- **Inconveniente:** la ejecución es algo más lenta que la de un lenguaje compilado directamente.

4.8. ¿Qué hace una máquina virtual?

- Actúa de **puente** entre el bytecode y el hardware concreto, traduciéndolo a instrucciones que el procesador entiende.
- Garantiza la **portabilidad** sin recompilar el código.
- **Gestiona la memoria** automáticamente: reserva espacio para los objetos y libera la que no se usa mediante el **recolector de basura**.
- **Verifica** el bytecode y protege la memoria de cada aplicación (seguridad y estabilidad).
- Interpreta el bytecode línea a línea o lo compila en tiempo real (**JIT**, Just-In-Time) para mejorar el rendimiento.

4.9. Java: del código a la ejecución

Java combina compilación e interpretación: primero se **compila** a bytecode y después una máquina virtual lo **ejecuta** en cada equipo.

Paso	Elemento	Qué ocurre
1	Código fuente	`Hola.java`: lo escribe el programador.
2	Compilador `javac`	Traduce el código fuente a bytecode.
3	Bytecode	`Hola.class`: código intermedio, igual en todos los sistemas.
4	JVM	Ejecuta el bytecode en cada equipo: lo interpreta o lo compila con JIT.

Sigla	Significado
JDK	**Java Development Kit:** para desarrollar. Incluye `javac`, otras herramientas y el JRE.
JRE	**Java Runtime Environment:** para ejecutar. Incluye la JVM y las bibliotecas estándar.
JVM	**Java Virtual Machine:** ejecuta el bytecode y gestiona la memoria.

> **IDEA CLAVE · Escribe una vez, ejecuta en cualquier sitio:** el mismo `.class` funciona con la JVM de cualquier sistema.

4.10. Frameworks y entornos de ejecución

> **DEFINICIÓN · Framework:** plataforma con bibliotecas y estructuras ya construidas para no partir de cero. Facilita el desarrollo rápido y la reutilización de código, pero crea dependencia: cambiar de framework obliga a reescribir buena parte de la aplicación. Ejemplos: .NET, Spring.

DEFINICIÓN · Entorno de ejecución (runtime): conjunto de utilidades que permiten ejecutar programas ya compilados: la máquina virtual más las bibliotecas de clases estándar (API). Actúa de intermediario entre el lenguaje y el sistema operativo. Ejemplo: JRE (Java Runtime Environment).

IDEA CLAVE · El framework acelera el desarrollo; el runtime hace posible la ejecución.

Glosario del bloque

Término	Definición
Traductor	Convierte alto nivel en ensamblador o código máquina.
Compilador	Traduce todo el programa antes de ejecutarlo.
Intérprete	Traduce y ejecuta línea a línea.
Preprocesado	Primera fase de compilación: procesa directivas como `#include` y `#define`.
Código objeto	Código máquina generado al compilar; aún sin las librerías.
Enlazador	Une el código objeto con las librerías para obtener el ejecutable.
Ejecutable	Programa final que se puede ejecutar sin compilador.
Bytecode	Código intermedio independiente del sistema.
Máquina virtual	Ejecuta el bytecode en cada equipo (ej. JVM).
Recolector de basura	Libera automáticamente la memoria que ya no se usa.
JIT	Compilación en tiempo de ejecución para mejorar el rendimiento.
javac	Compilador de Java: genera bytecode (`.class`).
JDK / JRE / JVM	Kit de desarrollo / entorno de ejecución / máquina virtual de Java.
Framework	Plataforma con estructuras y bibliotecas ya construidas.
Entorno de ejecución (runtime)	Máquina virtual + bibliotecas estándar para ejecutar programas (ej. JRE).

Para comprobar lo aprendido

1. ¿Qué diferencia hay entre un compilador y un intérprete?
2. Ordena las fases de la compilación y explica qué hace el enlazador.
3. ¿Por qué los errores de un programa interpretado aparecen más tarde que los de uno compilado?
4. Describe el recorrido de un programa Java desde `Hola.java` hasta su ejecución.
5. ¿Qué diferencia hay entre JDK, JRE y JVM?
6. Cita tres funciones de la máquina virtual.

5. Fases del desarrollo de una aplicación

5.1. Fases del desarrollo

> DEFINICIÓN · **Desarrollo de software:** proceso que lleva una aplicación desde la idea inicial hasta su puesta en funcionamiento y mantenimiento.

Sigue 8 fases, que pueden solaparse entre sí: **fase inicial, análisis, diseño, implementación, pruebas, explotación, mantenimiento y retirada**.

> DEFINICIÓN · **Requisito:** necesidad o condición que el software debe cumplir según el cliente.

5.2. Qué se hace en cada fase

Nº	Fase	Qué se hace
1	Fase inicial	Estudio de viabilidad: idea, costes, plazos y recursos necesarios.
2	Análisis	Se recogen y documentan los requisitos que necesita el cliente.
3	Diseño	Arquitectura, base de datos, interfaces y estructura de módulos.
4	Implementación	Se escribe el código y se integran los distintos módulos.
5	Pruebas	Se comprueba que cumple los requisitos y no tiene errores.
6	Explotación	Instalación, puesta en marcha y formación a los usuarios.
7	Mantenimiento	Corrección de errores y mejoras con el software en producción.
8	Retirada	Fin del soporte y migración de los datos a otro sistema.

5.3. Ciclos de vida del software

> DEFINICIÓN · **Ciclo de vida del software:** conjunto de fases por las que pasa un software desde que se concibe hasta que se retira.
>
> DEFINICIÓN · **Modelo de ciclo de vida:** forma de organizar y encadenar las fases del desarrollo.

Los modelos se agrupan en dos familias:

- **Clásicos (predictivos):** se planifica todo al principio. Cascada, cascada con realimentación, en V, iterativo incremental, prototipado y espiral.
- **Ágiles (adaptativos):** entregas cortas y frecuentes; el cambio es bienvenido. Scrum, Kanban, XP, Lean, Scrumban y DevOps.

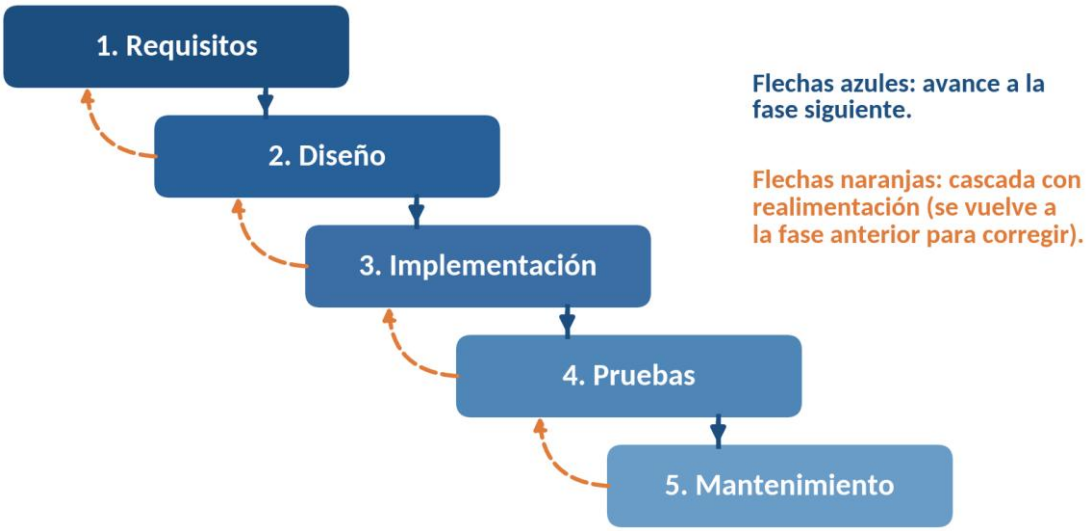
¿Cómo elegir? Depende de la estabilidad de los requisitos, el tamaño del proyecto, el nivel de riesgo y la implicación del cliente.

5.4. Modelo en cascada

> DEFINICIÓN · **Modelo en cascada:** las fases se suceden en orden estricto —requisitos, diseño, implementación, pruebas y mantenimiento—, sin vuelta atrás: no se empieza una fase hasta terminar la anterior.

- **Ventajas:** fácil de entender y gestionar; fases y entregables bien definidos; documentación en cada etapa.
- **Inconvenientes:** no hay vuelta atrás; el cliente ve el producto al final; los errores se detectan tarde.

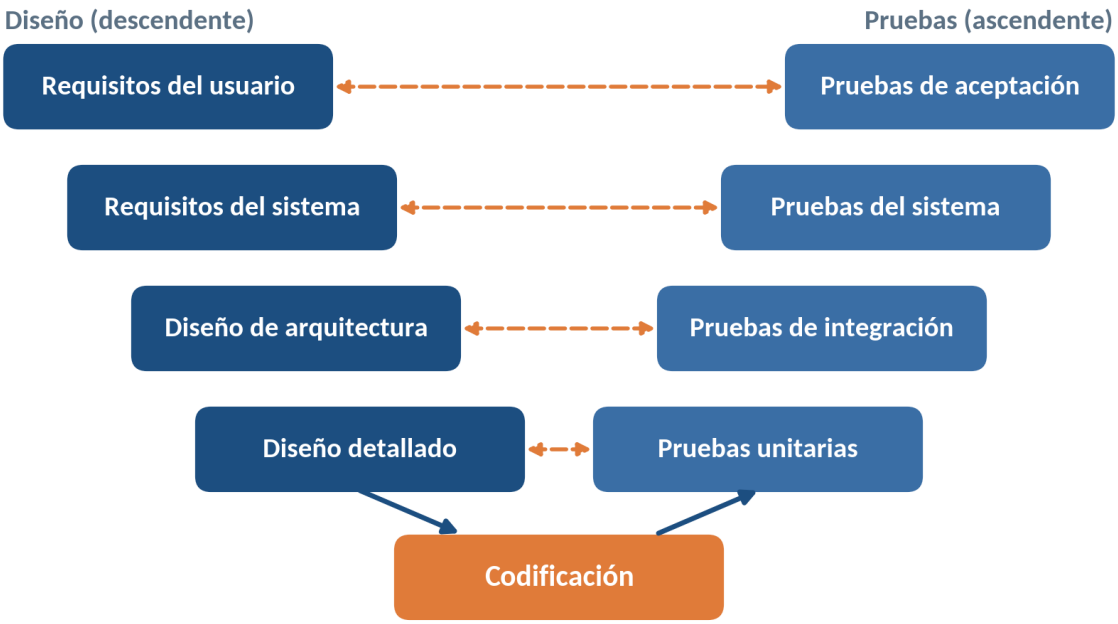
La **cascada con realimentación** permite volver a la fase anterior para corregir errores.



Modelo en cascada y cascada con realimentación

5.5. Modelo en V

> DEFINICIÓN · **Modelo en V:** variante de la cascada en la que cada fase de diseño tiene asociada una fase de pruebas que la valida.



Modelo en V: cada fase de diseño se valida con su fase de pruebas

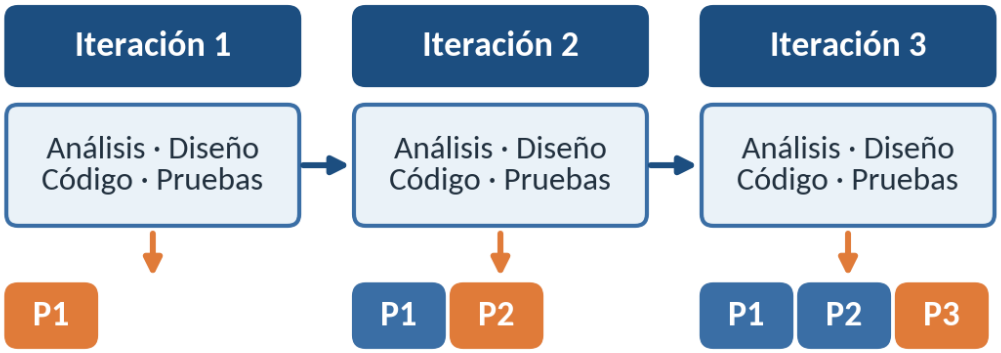
En el vértice está la **codificación**. Las pruebas se planifican desde el principio; es muy usado en software crítico, aunque es poco flexible ante cambios.

5.6. Modelo iterativo incremental y prototipado

DEFINICIÓN · **Modelo iterativo:** se repite el ciclo de desarrollo mejorando el producto en cada vuelta.

DEFINICIÓN · **Modelo incremental:** cada ciclo añade una parte funcional nueva al producto.

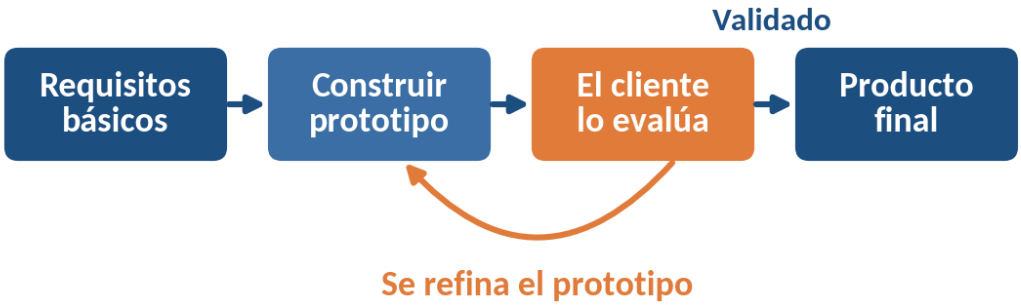
- Se divide el proyecto en pequeñas entregas; cada iteración incluye análisis, diseño, código y pruebas.
- El cliente ve resultados desde pronto y los errores y cambios se detectan antes.



Modelo iterativo incremental

DEFINICIÓN · **Prototipado:** se construye una versión preliminar del software para validar los requisitos con el cliente.

- **Prototipo desechable:** se tira tras validar los requisitos.
- **Prototipo evolutivo:** se mejora hasta convertirse en el producto final.
- Es útil cuando los requisitos no están claros. Riesgo: que el cliente crea que el prototipo ya es el producto.



Prototipado

5.7. Modelo en espiral

DEFINICIÓN · **Modelo en espiral:** modelo iterativo centrado en el análisis de riesgos, que combina prototipado y cascada. Cada vuelta de la espiral recorre cuatro actividades.

- **Objetivos:** se definen los objetivos y alternativas de la vuelta.
- **Análisis de riesgos:** se identifican y reducen los riesgos, a veces con prototipos.
- **Desarrollo y pruebas:** se desarrolla y valida el producto de esta vuelta.
- **Planificación:** se revisa el resultado y se planifica la siguiente vuelta.



Modelo en espiral: cada vuelta recorre las cuatro actividades

Es adecuado para proyectos grandes, costosos y de alto riesgo; es más complejo y caro de gestionar.

5.8. Manifiesto ágil

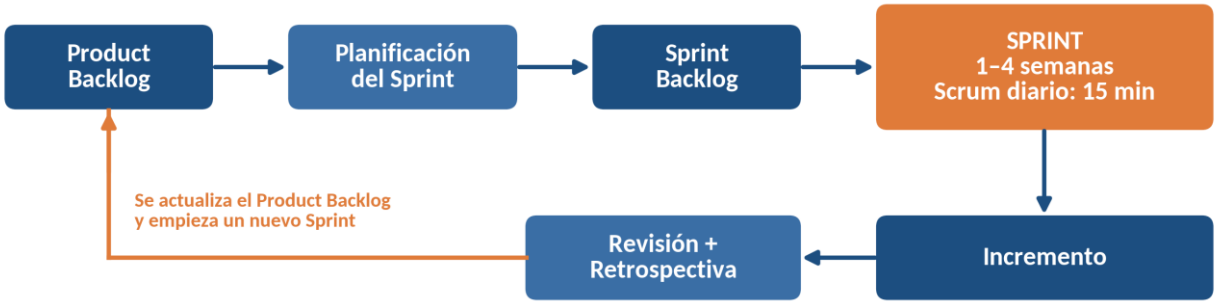
> DEFINICIÓN · **Manifiesto Ágil:** documento de 2001 que propone 4 valores: se valora lo de la izquierda por encima de lo de la derecha.

Se valora más…	…que
Individuos e interacciones	Procesos y herramientas
Software funcionando	Documentación exhaustiva
Colaboración con el cliente	Negociación contractual
Respuesta ante el cambio	Seguir un plan

Se concreta en 12 principios: entregas frecuentes, cambios bienvenidos, equipos autoorganizados, simplicidad y mejora continua.

5.9. Scrum: el ciclo del Sprint

> DEFINICIÓN · **Scrum:** marco de trabajo ágil que organiza el proyecto en **Sprints** (iteraciones de 1 a 4 semanas) y entrega un incremento al final de cada uno.



Ciclo de un Sprint en Scrum

5.10. Scrum: roles, eventos y artefactos

Elemento	Contenido
Roles	**Product Owner:** define y prioriza qué se construye. **Scrum Master:** facilita el proceso y quita obstáculos. **Equipo de desarrollo:** construye el producto, autoorganizado.
Eventos	Sprint (1–4 semanas), planificación del Sprint, scrum diario (15 min), revisión del Sprint y retrospectiva.
Artefactos	**Product Backlog:** lista priorizada de lo que se quiere. **Sprint Backlog:** tareas elegidas para el Sprint. **Incremento:** producto funcional entregado.

5.11. Kanban

> **DEFINICIÓN · Kanban:** método ágil que organiza el trabajo visualmente en un tablero de columnas (por ejemplo, Por hacer, En curso, Hecho) y limita el trabajo simultáneo.



Ejemplo de tablero Kanban con límite WIP = 2 en «En curso»

- **Origen:** Toyota; «kanban» significa tarjeta.
- **Visualizar** el trabajo en un tablero con tarjetas.
- **Limitar el WIP** (Work In Progress): máximo de tareas en curso a la vez.
- **Flujo continuo:** sin sprints ni fechas fijas.
- **Medir** el tiempo (lead time) y detectar cuellos de botella.

- Muy usado en mantenimiento y soporte.

> DEFINICIÓN · **WIP (Work In Progress):** trabajo en curso; en Kanban se fija un límite por columna para evitar acumular tareas a medias.

5.12. Otras metodologías ágiles

Metodología	Idea principal
XP	**Extreme Programming:** programación en parejas, pruebas antes del código (TDD), integración continua.
Lean	Nace en Toyota: eliminar desperdicio, entregar rápido y mejorar de forma continua.
Scrumban	Mezcla Scrum (roles, sprints) con Kanban (tablero y límites WIP).
DevOps	Une desarrollo y operaciones: automatiza integración, pruebas y despliegue (CI/CD).

5.13. Comparativa de modelos

Modelo	Cuándo usarlo	Ventaja	Inconveniente
Cascada	Requisitos claros y estables; proyectos pequeños	Sencillo y bien documentado	Rígido: no admite cambios
En V	Software crítico donde las pruebas son clave	Pruebas planificadas desde el inicio	Poco flexible ante cambios
Iterativo incremental	Proyectos medianos con entregas parciales	Resultados pronto y errores detectados antes	Exige una buena arquitectura
Espiral	Proyectos grandes, caros y de alto riesgo	Gestiona los riesgos desde el inicio	Complejo y costoso de gestionar
Scrum	Requisitos cambiantes; equipos pequeños	Entregas cortas y cliente implicado	Exige mucha implicación del cliente
Kanban	Soporte, mantenimiento y flujo continuo	Flexible y fácil de implantar	Sin plazos fijos: menos previsible

5.14. Documentación del proyecto

En cada fase del desarrollo se genera documentación:

- **Manual de usuario:** explica cómo usará el usuario la aplicación.
- **Manual técnico:** dirigido a programadores y administradores.
- **Manual de instalación:** detalla el proceso de instalación de la aplicación.

5.15. Pruebas y mantenimiento

Tipo de prueba	Descripción
Unitarias	Se prueba cada parte del software por separado.
De integración	Se comprueba que todas las partes funcionan bien juntas.
Beta test	Prueba final en el entorno real, con el cliente.

Mantenimiento	Descripción
Correctivo	Soluciona errores detectados.
Perfectivo	Mejora la funcionalidad ya existente.
Evolutivo	Añade nuevas funciones que pide el cliente.
Adaptativo	Adapta el software a nuevos entornos o tecnologías.

5.16. Roles en el desarrollo de software

Rol	Responsabilidad
Arquitecto de software	Decide cómo se estructura el proyecto y con qué tecnologías.
Jefe de proyecto	Dirige el equipo, gestiona los tiempos y trata con el cliente.
Analista	Estudia el problema y diseña el sistema.
Programador	Codifica y prueba los distintos módulos de la aplicación.

Glosario del bloque

Término	Definición
Fases del desarrollo	Fase inicial, análisis, diseño, implementación, pruebas, explotación, mantenimiento y retirada.
Requisito	Necesidad que debe cumplir el software.
Ciclo de vida	Fases desde que se concibe el software hasta su retirada.
Modelo predictivo	Se planifica todo al principio (cascada, V…).
Modelo ágil	Entregas cortas y adaptación al cambio (Scrum, Kanban…).
Cascada	Fases secuenciales sin vuelta atrás.
Modelo en V	Cada fase de diseño tiene su fase de pruebas.
Iterativo / incremental	Repetir ciclos / añadir partes funcionales.
Prototipo	Versión preliminar para validar requisitos.
Espiral	Iteraciones guiadas por el análisis de riesgos.
Sprint	Iteración de 1 a 4 semanas en Scrum.
Product Owner / Scrum Master	Prioriza qué se construye / facilita el proceso.
Backlog	Lista de trabajo pendiente (de producto o del sprint).
Kanban	Tablero visual con límite de trabajo en curso.
WIP	Trabajo en curso.
XP / Lean / DevOps	Otras metodologías ágiles.

Término	Definición
Prueba unitaria / de integración / beta	Por partes / en conjunto / en entorno real con el cliente.
Mantenimiento correctivo / perfectivo / evolutivo / adaptativo	Arreglar errores / mejorar lo existente / añadir funciones / adaptar a nuevos entornos.
Manual de usuario / técnico / de instalación	Para quien usa / para quien programa o administra / para quien instala.

Para comprobar lo aprendido

1. Enumera las 8 fases del desarrollo de una aplicación.
2. ¿Qué diferencia un modelo predictivo de uno ágil?
3. ¿Qué roles y eventos tiene Scrum?
4. Explica qué es un tablero Kanban y qué significa limitar el WIP.
5. ¿Cuándo elegirías cascada y cuándo Scrum? Justifícalo.
6. Distingue mantenimiento correctivo, perfectivo, evolutivo y adaptativo.

Correspondencia con los criterios de evaluación

CE	Criterio	Apartados
CE1.a	Relación de los programas con los componentes del sistema: memoria, procesador, periféricos	Bloque 1
CE1.b	Clasificación de los lenguajes de programación	3.1 – 3.9
CE1.c	Diferenciación entre código fuente, objeto y ejecutable	3.10 – 3.11
CE1.d	Generación de código intermedio para su ejecución en máquinas virtuales	4.7 – 4.10
CE1.e	Funcionalidad de las herramientas utilizadas en programación	2.2 – 2.8
CE1.f	Funcionamiento de los distintos tipos de traductores de lenguajes	4.1 – 4.6
CE1.g	Fases de desarrollo de una aplicación informática	Bloque 5

Contenidos adaptados y simplificados a partir de material propio, de la presentación «UA1. Desarrollo de software» de Luis del Moral Martínez (CC BY-NC-SA 4.0) y de los apuntes de Entornos de Desarrollo de Javier Gutiérrez.